

Saint Mary's University

PROJECT REPORT

MCDA 5585

Taken in Lieu of:

MCDA 5520

Submitted By:

Francis Alex Kuzhippallil

A00463084

under the guidance of:

Dr. Pawan Lingras

Dr. Adel Merabet

in partial fulfillment for the awarding of the degree:

Master of Science in

Computing and Data Analytics

September 2022 - December 2023

TABLE OF CONTENTS

TABLE OF CONTENTS.....	1
LIST OF FIGURES.....	2
LIST OF TABLES.....	2
CERTIFICATE	3
ACKNOWLEDGEMENT.....	4
EXECUTIVE SUMMARY	5
PROJECT BACKGROUND.....	6
1. Company Background	6
2. Team Composition	6
3. My Role and Responsibilities	6
PROJECT OVERVIEW.....	7
PROJECT GOALS	7
LEARNING GOALS.....	8
PROJECT SCOPE.....	8
ASSUMPTIONS	9
REQUIREMENTS GATHERING	9
1. Software Requirements	9
2. Hardware Requirements.....	10
PLANNING METHODOLOGY	11
LITERATURE SURVEY	11
IMPLEMENTATION	12
1. Planogram Compliance	12
2. Tag Compliance.....	16
RESULTS	19
1. Planogram Compliance	19
2. Tag Compliance.....	20
ACHIEVEMENTS.....	22
CHALLENGES	22
PROJECT GOALS REVIEW.....	22
LEARNING GOALS REVIEW	23
CONCLUSION.....	24
1. Summary	24
2. Work Distribution.....	24

FUTURE WORK	25
REFERENCES	26
APPENDIX	27
GLOSSARY.....	27
1. List of Abbreviations	27
2. List of Images	27

LIST OF FIGURES

Figure 1: Planogram Compliance Flowchart	12
Figure 2: Tag Compliance Pipeline	16
Figure 3: Performance Measure	19
Figure 4: Cls Loss	19
Figure 5: Performance Measure	21
Figure 6: Cls Loss	21
Figure 7: Storing images.....	27
Figure 8: Row-wise stitched image (Row 1).....	27
Figure 9: Planogram generated	28
Figure 10: Custom tag.....	28

LIST OF TABLES

Table 1: List of Learning Goals	8
Table 2: Jetson Nano Specifications.....	10
Table 3: Jetson Tx2 Specifications.....	10
Table 4 - Hardware Requirements	10
Table 5: Working Environment	11
Table 6: Yolov5 Performance (Planogram Compliance)	19
Table 7: Planogram Compliance Testcase.....	20
Table 8: Yolov5 Performance (Tag compliance)	20
Table 9: Tag Compliance Test Cases	21
Table 10: Project Goals Completion Status.....	22
Table 11: Learning Goals Completion Status	23
Table 12: Hours Spent.....	24
Table 13: List of Abbreviations.....	27

CERTIFICATE

This is to certify that **Mr. Francis Alex Kuzhippallil (ID: A00463084)** a student at Saint Mary's University in the Master of Science in Computing and Data Analytics Program has completed the projects in fulfillment of the course requirements.

.....
Dr. Pawan Lingras

ACADEMIC SUPERVISOR

DEPARTMENT OF MATHEMATICS

AND COMPUTING SCIENCE

SAINT MARY'S UNIVERSITY

923 Robie St. Halifax, NS, Canada B3H 3C3

.....
Dr. Adel Merabet

PROJECT SUPERVISOR

DIVISION OF ENGINEERING

SAINT MARY'S UNIVERSITY

923 Robie St. Halifax, NS, Canada B3H 3C3

ACKNOWLEDGEMENT

I like to take the opportunity to thank all the people who helped me.

Firstly, I would like to express my sincere gratitude to **Dr. Adel Merabet** for his valuable guidance, continuous support throughout the project. I am highly indebted to his constant supervision. I would also like to thank **Dr. Pawan Lingras** for his enthusiasm in providing me with the opportunity and sharing valuable guidance throughout the project. I thank **Mrs. Kannagi Mahajan, Mr. Dan Penny, Mr. Neil Williams, Mrs. Ravee Kadam, and Mrs. Neelam Pal** for helping with the project report.

EXECUTIVE SUMMARY

This report elucidates the project work that I completed under the guidance of Dr. Adel Merabet from September 2022 to January 2023. This project focuses on how I automated two main activities conducted at retail companies on a day-to-day basis: planogram and tag compliance. Planogram is the blueprint of the shelves, which provides information about the products present on the shelves and their respective positions on the shelves. Planogram compliance involves verifying if the arrangement of the products on the shelves complies with the planogram. Tags refer to the labels that provide a description (name, price, weight, barcode, etc.) of the products on the shelf. Tag compliance involves verifying if the tag content's position matches the database information.

At present, most retail companies perform the planogram and tag compliance manually. The limitations of this approach are the increased risk of human errors, time consumption, and manual effort. I aimed to provide an automated solution using the autonomous robot prototype to undertake planogram and tag compliance swiftly, effectively, and cost-efficiently. I had weekly meetings with Dr. Adel Merabet to discuss the progress at each stage, note the valuable suggestions provided, and perform full-fledged research to create an end-to-end automated pipeline for planogram and tag compliance. At the end of this project, Dr. Adel and I presented a live demonstration of the entire automated planogram and tag compliance pipeline to the committee from the business school, who was impressed with the progress made and provided feedback for the future.

The report consists of a project background and an overview to provide an understanding of the project I have worked on and the tasks assigned to me. The report further explains my contribution to the project, which includes the requirements I gathered, the methodology adopted, the literature survey conducted, the pipeline implemented (detailed explanation of planogram and tag compliance), and the results achieved for the planogram and tag compliance during my work. The report concludes with my achievements during my work, challenges faced, project goals review, learning goals review, conclusion, future work, references, appendix, and glossary.

PROJECT BACKGROUND

1. Company Background

The engineering division offers the first two years of an undergraduate engineering program. Faculty members within the engineering division supervise students from various disciplines to produce high-quality research work to publish in reputable conferences and journals.

2. Team Composition

The project was supervised and supported by Dr. Adel Merabet. The project team consisted of the below members.

- ❖ Dr. Adel Merabet (project guide)
- ❖ Mohammed Rahimipour (research assistant: focused on robot structure and navigation)
- ❖ Francis Alex Kuzhippallil (research assistant: focused on robot's software functionalities capabilities enhancement)

3. My Role and Responsibilities

My role: to work as a research assistant for Dr. Adel Merabet's project and utilize my image processing, machine learning, deep learning, and research skills to enhance the software capabilities of the autonomous robot.

My responsibilities: responsibilities assigned to me by Dr. Adel include.

- ❖ To gain an in-depth understanding of the inventory management process.
- ❖ To perform a literature review about planogram and tag compliance and understand the existing works performed in this domain.
- ❖ To get familiar with the components inside the autonomous robot prototype and obtain a basic knowledge of robot architecture.
- ❖ To learn more about Nvidia Jetson Nano and Nvidia Jetson Tx2. Also, gain hands-on experience on Nvidia Jetson Nano.
- ❖ To gain expertise in embedding the code in Nvidia Jetson Nano and ensuring the robot performs the task intended.
- ❖ To learn more about the cv2 library and its capabilities in image processing.
- ❖ To learn how to perform object detection and explore the possible methods to achieve the same.
- ❖ To create a pipeline to ensure planogram and tag compliance pipelines are automated.
- ❖ To perform compliance testing on various scenarios to check if the solution devised is robust and efficient.

PROJECT OVERVIEW

Brief history: inventory management is one of the many domains with enormous scope for implementing enhanced automated processes. SMU business school had funded a research project led by Dr. Adel Merabet to build a robot that performs inventory management tasks (planogram compliance, tag compliance, product stock detection, etc) automatically. Dr. Adel recruited me as a research assistant to use my skills to automate both planogram and tag compliance tasks using the robot.

Planogram: planogram is the blueprint of the shelves, which provides information about the products present on the shelves and their respective positions on the shelves. A retail organization's sales get affected by tag and planogram compliance. Planogram compliance involves verifying if the generated planogram by the robot matches the actual planogram.

Tag: tags refer to the product labels that describe name, price, weight, barcode, etc. Tag compliance involves matching the tag content and the information in the database.

Need for automation: performing planogram and tag compliance in conventional methods is daunting, with a high risk of errors and increased time consumption. My task in this project was to create an end-to-end planogram compliance and tag compliance pipeline and automate the process using an autonomous robot prototype as a proof of concept, to reduce human intervention and errors.

PROJECT GOALS

Provided below are the high-level project goals.

- ❖ Establish a planogram compliance pipeline, including detailed descriptions of each step.
- ❖ Establish a tag compliance pipeline, including detailed descriptions of each step.
- ❖ Implement end-to-end planogram compliance pipeline.
- ❖ Implement end-to-end tag compliance pipeline.
- ❖ Collaborate with Mohammed to embed the relevant Python scripts in Nvidia Jetson Nano to automate the planogram and tag compliance using the autonomous robot prototype.
- ❖ Assist Dr. Adel Merabet in drafting content for research paper publication in a suitable conference or journal paper.

LEARNING GOALS

Provided below in tabular format are the high-level learning goals.

Table 1: List of Learning Goals

No	Goals	Tasks
1	Planogram generation	Learn how an actual planogram is available.
		Create a planogram sketch for our customer racks in the lab.
2	Planogram generation	Apply image stitching to stitch images row-wise
		Applying vertical stitching/concatenation to generate the planogram
3	Planogram compliance	Apply object detection on the custom retail products dataset created using yolov5 and check if the extracted planogram complies with the actual planogram.
4	Tag compliance	Apply the OCR extraction technique using Python to extract necessary information from the tags, which is needed to check if the tag information matches with the database information.
5	Insights and feedbacks	Gain valuable guidance, insights, and valuable feedback from Dr. Adel Merabet, and Mohammed to complete the project.

PROJECT SCOPE

Below are the action items I worked on as part of the project scope and those beyond my scope of work.

In-Scope:

- ❖ Establish a planogram compliance pipeline, including detailed descriptions of each step.
- ❖ Design the sketch of the planogram, providing a blueprint of the items on the shelves in the testing environment (Dr. Adel Merabet's engineering lab).
- ❖ Create custom tags for each item on the shelves in the testing environment.
- ❖ Create a database to store relevant information regarding the items on shelves in the testing environment.
- ❖ Train the yolov5 model to detect the tags and items on the shelves in the testing environment.

Out of -Scope:

- ❖ Training the yolov5 model for all items in a retail store.
- ❖ Performing mock run of the robot performing planogram and tag compliance autonomously in a retail store.

ASSUMPTIONS

Below are the assumptions I made while performing my work on the project.

- ❖ All hardware components of the robot are intact and in working condition.
- ❖ A robot's navigation map is created and tested to ensure the robot follows the expected path.

REQUIREMENTS GATHERING

The preliminary step which I did for the project was requirements gathering. During initial meetings, Dr. Adel Merabet mentioned what he expects from the planogram and tag compliance tasks.

Planogram compliance: below are the requirements Dr. Adel Merabet provided.

- ❖ The robot will navigate through the lab based on the map designed.
- ❖ The coordinates at which the robot should trigger the camera to switch on and off are to be determined. The cameras in the robot should capture the items on the shelves.
- ❖ Design a planogram for the shelves in the lab (Dr. Adel's engineering lab). The planogram generated by the robot should match the planogram designed.
- ❖ The robot should automate the entire planogram compliance process.

Tag compliance: below are the requirements Dr. Adel Merabet provided.

- ❖ The robot will navigate through the lab based on the map designed.
- ❖ The coordinates at which the robot should trigger the camera to switch on and off are to be determined. The cameras in the robot should capture the tags on the shelves.
- ❖ Verification of tag contents and database information is required. Users should be able to view the status of compliance.
- ❖ The robot should automate the entire tag compliance process.

Below are the software and hardware requirements that I found essential for the project.

1. Software Requirements

Provided below are lists of software requirements.

- ❖ **Visual Studio Code**
- ❖ **Google Colab**

- ❖ Google Drive
- ❖ Python Libraries
- ❖ Microsoft SQL Server

2. Hardware Requirements

Provided below are lists of hardware requirements.

- ❖ **Nvidia Jetson Nano:** is a powerful mini-computer mainly used for computer vision applications such as image segmentation, object detection, live video capture, etc. For this project, I used Jetson Nano to execute live video capture from multiple cameras, extracting the frames from the video and transferring the image folders to another remote host. Table. 3 specifies the Jetson Nano specifications used for the project.

Table 2: Jetson Nano Specifications

CPU	Quad-Core ARM A57 @ 1.43 GHz
GPU	128 Core Maxwell
Storage	microSD
Memory	4 GB 64-bit LPDDR4 25.6 GB/s
Camera	2x MIPI CSI-2 DPHY Lanes
USB	4x USB 3.0, USB 2.0 Micro-B

- ❖ **Nvidia Jetson Tx2:** is also a powerful mini-computer faster than the Nvidia Jetson Nano, used for computer vision applications such as image segmentation, object detection, live video capture, etc. Tx2 controlled the robot's autonomous navigation. Table. 4 specifies the Jetson Nano specifications used for the project.

Table 3: Jetson Tx2 Specifications

CPU	Dual-Core Nvidia Denver 2 64-Bit CPU, Cortex A57 MP
GPU	256 Nvidia Pascal GPU architecture
Storage	32GB eMMC 5.1
Memory	8 GB 128-bit LPDDR4 Memory, 1866 MHx

- ❖ **Personal Computer:** table. 5 provides my personal laptop specifications. I used my device for research and implementation purposes.

Table 4 - Hardware Requirements

Operating System	Windows11
GPU	GTX 1080Ti
Web Browser	Chrome, Edge, Firefox etc.
Other Requirements	Internet Access

- ❖ **Turtlebot:** is the robot used for planogram and tag compliance. It follows ROS standards.

PLANNING METHODOLOGY

Working Environment

Table 5: Working Environment

Methodology adopted	Agile Methodology
Project Supervisor	Dr. Adel Merabet
Author	Francis Alex Kuzhippallil

I worked under constant supervision, valuable guidance, and corrections from Dr. Adel Merabet.

Brainstorm: we had weekly meetings to discuss progress made, challenges faced, and plans for next week. Feedback provided by Dr. Adel was noted and addressed.

Design: spend time and designed pipelines for tag compliance and planogram compliance.

Development: created end-to-end code for planogram and tag compliance pipelines based on the design created.

Quality Assurance: I tested different use cases for planogram and tag compliance to ensure the quality of the pipelines (results provided in the result section).

Deployment: Dr. Adel and I conducted a live demo with the business school director, and he was impressed with the results obtained. Furthermore, works are ongoing to test the robot in a real-time retail store.

LITERATURE SURVEY

Below are a few research papers I explored for the project.

[1] Varol used multiple approaches to execute planogram compliance and noted that the least computationally intensive model was the support vector machine. Varol achieved an object detection accuracy of 69% and further improved the accuracy to 81% with slight modifications in the height and width of the images.

[2,3] Parameswaran and Minu focused on implementing the detection of items on a shelf, estimation of space on the shelf, and identification of any change between two images taken at different times. Both authors used image differencing, principal component analysis, change vector analysis, and Kauth-Thomas transformation to detect differences between multiple images.

[4-7] Hinton, Ciregan, Lawrence, and Chong utilized deep learning techniques to ensure planogram compliance. Deep learning techniques are more robust and accurate if the dataset

is large. They used convolution neural networks for object detection of products on the shelf. The overall prediction accuracy was high.

[8] Girshick uses r-cnn and fast r-cnn techniques that are variants of convolution neural networks. R-CNN uses a region-based extraction technique which is much faster and more accurate than other convolution neural networks but computationally expensive.

IMPLEMENTATION

Below are the details of the design and execution phase. I used Python to implement both the planogram and tag compliance pipeline.

1. Planogram Compliance

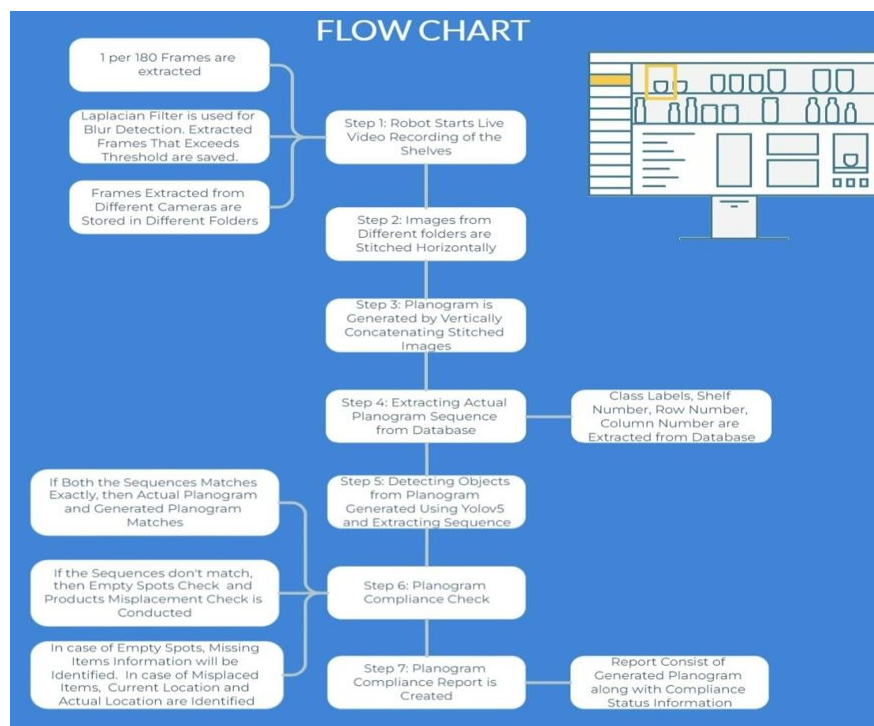


Figure 1: Planogram Compliance Flowchart

❖ Live video capture

Functionality description: this functionality involves the robot cameras capturing the frames (images) of the shelves while in motion.

How I implemented this functionality: firstly, I worked on extracting frames (images) during the robot's live video capture using the cv2 library. Then, I performed multiple runs of live video capture using the robot at different speeds and distances from the shelves to determine the optimal speed and distance to extract quality frames. Further, I extracted specific frames during live video capture to eliminate redundant frames. I tried to determine the ideal frame extraction ratio. After several test runs, I observed one frame per one-eighty frames is ideal

(Note: one second produces thirty frames). Then, I added a feature to detect and remove blurred frames using the Laplacian filter. The Laplacian filter is a second-order derivative filter mainly used for edge detections. Laplacian filter checks for blur based on the variance of the image. If an image contains low variance below a predefined threshold, then the image is declared blurred, as low-variance images have minuscule edges. After extracting quality frames, I stored the frames extracted in specific folders (frames captured from each camera focused on each row of the shelves are in specific folders). Fig. 7 (in the glossary section) represents a folder storing frames extracted from a single camera focused on row 1 of the shelves. Then, I worked on clearing the three folders before the execution of capturing new frames. Lastly, I reduced the computational overhead of Nvidia Jetson Nano by sending the image folders to another remote host via the scp command for further processing.

❖ Image stitching

Functionality description: this functionality involves row-wise (each folder containing extracted frames corresponds to a row on the shelf) horizontal stitching of extracted frames to create a panoramic image.

How I implemented this functionality: firstly, I researched the concept behind an image or panoramic stitching and found that image stitching involves four steps¹: detection of points, local invariant descriptors extraction from images, matching the two image descriptors, stitching images in a loop, creation of homography matrix using the Ransac algorithm, and performing warping transformation using the homography matrix. Further, I researched the different image stitching techniques and learned about cv2 and ORB. I implemented both techniques in Python and tested them on images from Google. I observed that cv2 performs better compared to ORB. Then, I worked on the implementation to read the shelves images from the folder and perform image stitching using the cv2 library. Fig. 8 (in the glossary section) represents the panoramic image of the first row of shelves.

❖ Concatenation of stitched images

Functionality description: this functionality involves vertical concatenation of the row-wise stitched images.

How I implemented this functionality: to generate a planogram that depicts the status of the shelves, I used vertical concatenation using the Python vconcat function. Note: I chose vertical concatenation as each camera focused on capturing the entire row. Hence each stitched image represents a single row of shelves. An example of a planogram generated (fig. 9) is in the glossary section.

¹ <https://medium.com/analytics-vidhya/panorama-formation-using-image-stitching-using-opencv-1068a0e8e47b>

❖ Planogram sequence extraction from database

Functionality description: this functionality involves extracting relevant product information (name, qty, price, and location) from multiple tables in the database to utilize the same for compliance (upcoming step).

How I implemented this functionality: firstly, I determined the number of tables required to store product information on the shelves, the schema of each table, keys and constraints in the tables, and dependencies between each table. Then, I created the database tables in Microsoft SQL Server and inserted records into the tables. Further, I worked on the implementation to extract relevant information from the database and store the information in variables. This step is relevant because I perform planogram compliance based on pattern matching.

❖ Object detection

Functionality description: this functionality detects products from the planogram generated (step 3: concatenation of images) and extracts relevant information about the detected products.

How I implemented this functionality: firstly, I explored the object detection concept, the steps involved, and its applications. Then, I created a custom dataset from Google images suitable for the haar cascade classifier, implemented object detection using the haar cascade classifier, debugged code, fixed errors, and evaluated the model's performance. Next, I created a custom dataset from Google images for the yolov5 classifier, implemented object detection using the yolov5 (high-speed and accurate predefined convolutional neural network object detection algorithm) classifier, debugged code, fixed errors, and evaluated the model's performance. Then, I finalized proceeding with the yolov5 classifier for object detection as it performs better than the haar classifier.

Then, I created a custom image dataset of the products on the shelves suitable for yolov5 model training. Then, I split the dataset into training and testing sets. Due to limited data, I increased the dataset volume using a data augmentation technique. Then, I performed the yolov5 model training, evaluated the model's performance, and executed multiple test cases (all items present, a few misplaced items, and a few items missing from the shelves). Further, I performed dataset annotation annotating the images and extracting the bounding box coordinates with each product label in yolov5 format using MakeSense AI².

Further, I trained the yolov5 model for 200 epochs and then fed the planogram generated as an input to the yolov5 algorithm to detect the objects. The algorithm detected the object's class label and bounding box accurately. I stored the class label of the detected objects and their bounding box coordinates in a text file to generate the sequence of the status of racks.

² <https://www.makesense.ai/>

❖ Planogram compliance check

Functionality description: this functionality checks if the generated planogram information complies with the actual planogram information.

How I implemented this functionality: this functionality involves checking if the existing planogram sequence matches the actual planogram sequence. Below are the three use cases I brainstormed and implemented.

Missing items check: it is checked based on the length of both sequences.

Missing items = yes {if length (actual sequence) != length (current planogram sequence)}

Incorrect positions check: there are misplaced products if any violation occurs at any index when the two sequences generated are mapped one-to-one.

Perfect Match: the current planogram complies perfectly with the actual planogram if missing or misplaced items are not found.

I tested the above compliance use cases under multiple test cases and observed that the compliance functionality performs as expected. The results of the test cases conducted are in the results section.

❖ Report generation

Functionality description: this functionality generates a report with a timestamp to provide a detailed report about the compliance status. A sample report is in the appendix section.

How I implemented this functionality: firstly, I explored how to generate a report (word or pdf) using Python. Then, I worked on the implementation of creating a new folder to store the report with timestamps on each execution. Further, I prepared the content to display on the report for the three use cases compliance check (perfect match, misplaced items, missing items). In case of missing items, those class labels present in the actual sequence and not present in the current planogram sequence are found and displayed along with their positions on the shelf. In case of misplaced items, then the product's current and actual position are displayed in the report. In case of a perfect match, a success message that the planogram generated matches the actual planogram is in the report.

Lastly, I automated the robot to perform all the tasks (frame extraction, image stitching, image concatenation, yolov5 object detection, compliance, and report generation). I observed the time taken for the robot to perform the five tasks, performed multiple test cases to identify bugs in the code, and fixed the bugs.

2. Tag Compliance

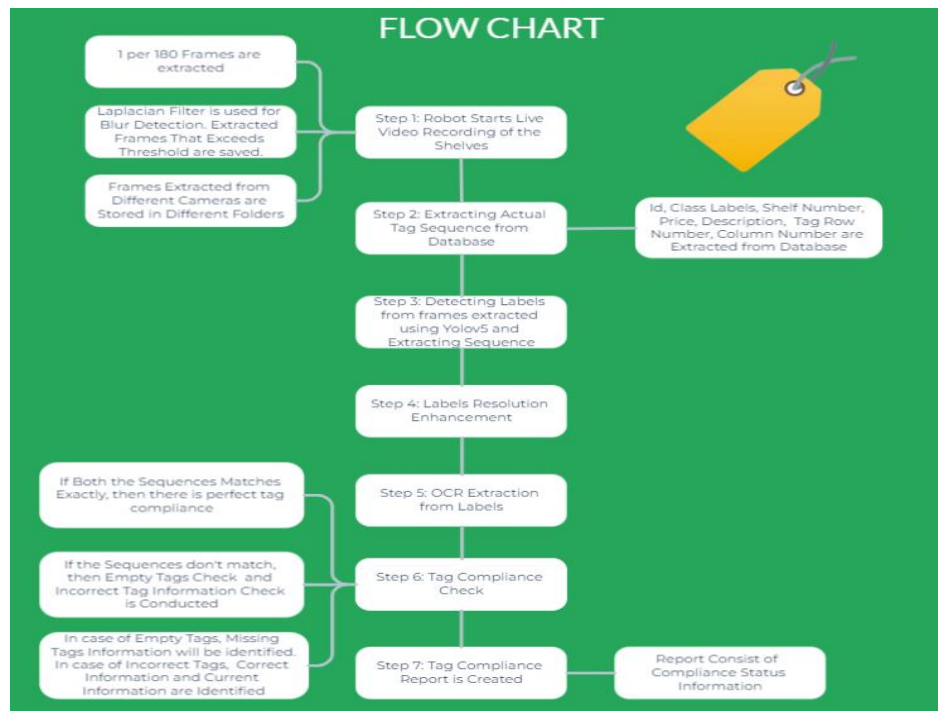


Figure 2: Tag Compliance Pipeline

The second pipeline I implemented using Python was tag compliance. Tag compliance scope involves checking if the tag information matches the information in the database (content match, positions match, and missing tags). I created custom tags for each product on the shelves. The custom tag content includes product name, price, weight, and id. Fig. 10 (in the glossary section) depicts a custom tag.

Note: live video capture and report generation from the tag compliance are the same as in planogram compliance. I skipped those steps to avoid tautology.

❖ Database - sequence extraction

Functionality description: this functionality involves extracting relevant tag information (product name, product weight, product price, and tag position) from multiple tables in the database to utilize the same for compliance (upcoming step).

How I implemented this functionality: firstly, I determined the number of tables required to store product information on the shelves, the schema of each table, keys and constraints in the tables, and dependencies between each table. Then, I created the database tables in Microsoft SQL Server and inserted records into the tables. Further, I worked on the implementation to extract relevant information from the database and store the information in variables. This step is relevant because I perform tag compliance based on pattern matching.

❖ Tags Detection

Functionality description: this functionality detects labels from the images extracted during live video capture.

How I implemented this functionality: I used yolov5 to detect tags from the images extracted during live video capture. Then, I created a dataset from the custom tags I made of the products on the shelves suitable for yolov5 model training. Then, I split the dataset into training and testing sets. Due to limited data, I increased the dataset volume using a data augmentation technique. Then, I performed the yolov5 model training and evaluated the model's performance. Further, I performed dataset annotation annotating the images and extracting the bounding box coordinates with each product label in yolov5 format using MakeSense AI³.

Further, I trained the yolov5 model for 200 epochs and then fed the images which contain tags as an input to the yolov5 algorithm to detect. The algorithm detected the tag's class label and bounding box accurately. Then, I extracted the region of interest (bounding box area) and saved it as an image in a folder.

❖ Tags resolution enhancement

Functionality description: this functionality focuses on the resolution enhancement of the tags detected by the yolov5 model. This step is a prerequisite before OCR extraction to obtain accurate extraction of texts from the image.

How I implemented this functionality: firstly, I explored the concept behind image resolution enhancement and searched for different techniques. I learned that EDSR is a powerful, fast, and efficient residual network architecture I used for image resolution enhancement. It has different scaling factors (2,3,4) to provide various resolutions.

I worked on the EDSR technique and tested the performance on images from Google. I observed that EDSR had enhanced the input images significantly. Then, I tried the technique on multiple scaling factors (2,3,4) on the tags detected using the yolov5 model and obtained enhanced resolution tags suitable for OCR extraction. I observed that on a scaling factor of 2, the technique performed the best. Hence for this project, the scaling factor was finalized on 2.

❖ OCR extraction from labels

Functionality description: this functionality involves extracting texts from images and storing the information for tag compliance checks.

How I implemented this functionality: firstly, I explored the concept behind OCR extraction, the techniques available, and an in-depth study of each available technique. I tested the accuracy of OCR extraction using easyOCR and py-tesseract OCR extraction and noticed that

³ <https://www.makesense.ai/>

easyOCR performs better. EasyOCR is also a deep-learning library used for extracting text from images. It can detect across 40 languages. Further, I worked on the implementation of OCR extraction using easyOCR.

❖ Tag compliance check

Functionality description: this functionality checks if the tags information complies with the information in the database.

How I implemented this functionality: this functionality involves checking if the sequence and information of tags match the actual tag sequence and information. Below are the four use cases I brainstormed and implemented.

Missing Tags Check: the first check made is the missing tags check. It is checked based on the concept of the length of both sequences.

Missing items = Yes {if length (actual sequence) != length (current tag sequence)}

In case of missing tags, class labels and product names present in the actual sequence and not present in the current tag sequence are found and displayed along with their positions in the report.

Incorrect Positions Check: there are misplaced tags if any violation occurs at any index when two sequences generated are mapped one-to-one. In case of any violation, the tag's actual and current position is in the report.

Tag Contents Check: there are incorrect tags if any violation occurs when the two sequences generated are mapped one-to-one and checked if the contents (name, price, and quantity) match. In case of any violation, the tag's actual and current content is in the report.

Perfect Match: the current tags comply perfectly with actual tags if there are no missing, incorrect, or misplaced tags. A success message that the tags comply perfectly with the database information shown in the report.

Lastly, I automated the robot to perform all the tag compliance tasks. I observed the time taken for the robot to perform the tasks, performed multiple test cases to identify the bugs in the code, and fixed the bugs.

RESULTS

This section consists of two parts. The first part constitutes the results obtained for planogram compliance. The latter part constitutes tag compliance results.

1. Planogram Compliance

Below are the yolov5 model's performance evaluation and planogram compliance test cases. The preconditions under which I executed the test cases were as follows:

- ❖ The robot operates at base speed.
- ❖ Captured images using three cameras installed on the robot.
- ❖ The frame extraction ratio is one frame per one-eighty frames.

Table 6: Yolov5 Performance (Planogram Compliance)

ID	Epochs	Precision	Recall	F-Measure	Classification Loss
1	50	0.88	0.50	0.64	0.043
2	150	0.86	0.96	0.91	0.013
3	200	0.98	0.99	0.98	0.006

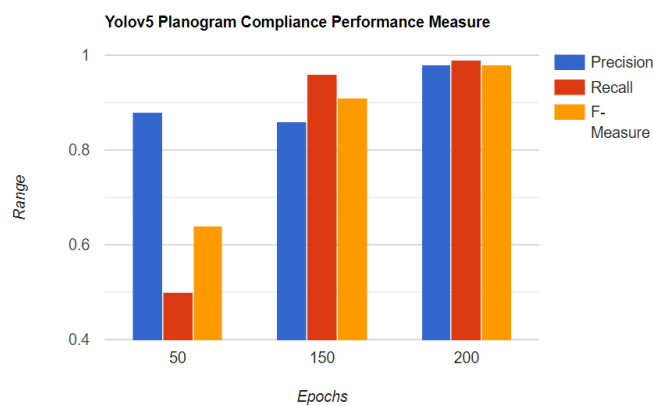


Figure 3: Performance Measure

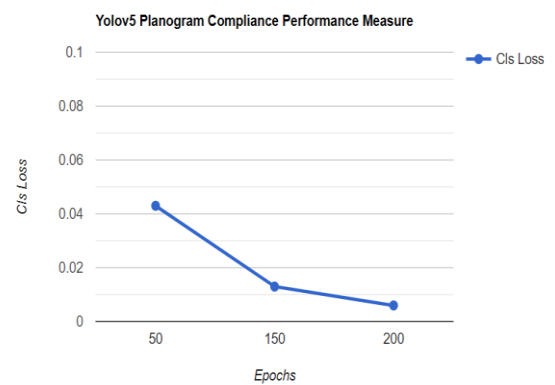


Figure 4: Cls Loss

The performance measure denotes that at the end of 200 epochs, the yolov5 model can predict all objects accurately with high precision, recall, f-measure, and minimal classification loss.

Table 7: Planogram Compliance Testcase

ID	Scenario	Object Detection Status (Detected/Not detected)	Expected Planogram Compliance Status (Matches/Not matches)	Actual Planogram Compliance Status (Matches/Not matches)	Total Execution Time (Min) (approx.)	Test Case Result
1	All products are present and in their correct positions	Detected	Match	Match	37 min	Pass
2	All products are present but multiple products are misplaced	Detected	No match	No match	40 min	Pass
3	There are empty items present on the shelf	Detected	No match	No match	39 min	Pass

The link to the reports is in the appendix section.

2. Tag Compliance

Below are the yolov5 model's performance evaluation and tag compliance test cases. The preconditions under which I executed the test cases were as follows:

- ❖ The robot operates at base speed.
- ❖ Captured images using three cameras installed on the robot.
- ❖ The frame extraction ratio is one frame per one-eighty frames.

Table 8: Yolov5 Performance (Tag compliance)

ID	Epochs	Precision	Recall	F-Measure	Classification Loss
1	50	0.84	0.46	0.59	0.031
2	150	0.90	0.96	0.93	0.003
3	200	0.94	0.98	0.96	0.002

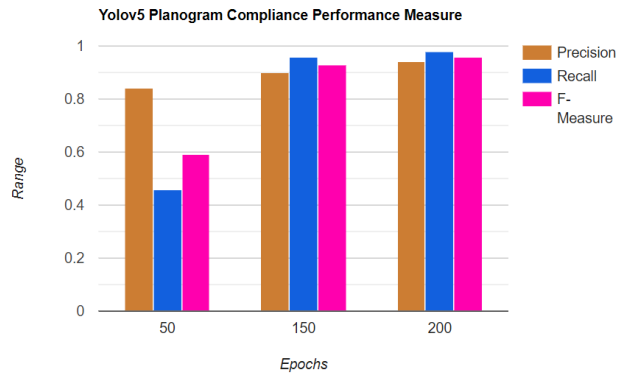


Figure 5: Performance Measure

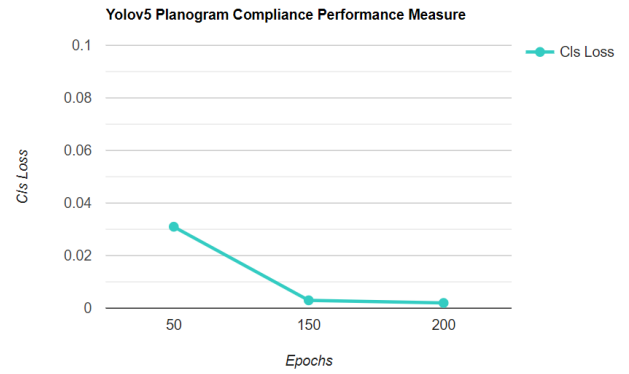


Figure 6: Cls Loss

The performance measure denotes that at the end of 200 epochs, the yolov5 model can predict all objects accurately with high precision, recall, f-measure, and minimal classification loss.

Table 9: Tag Compliance Test Cases

ID	Scenario	Object Detection Status (Detected/Not detected)	Expected Tag Compliance Status (Matches/Not matches)	Actual Tag Compliance Status (Matches/Not matches)	Total Execution Time (Min) (approx.)	Test Case Result
1	All tags are present and in their correct positions and the content of the tags is correct	Detected	Match	Match	20 min	Pass
2	All tags are present but multiple tags are misplaced	Detected	No match	No match	24 min	Pass
3	There are empty items present on the shelf	Detected	No match	No match	22 min	Pass
4	All tags are present at the right positions, but tags have incorrect information.	Detected	No match	No match	24 min	Pass

The link to the reports is in the appendix section.

ACHIEVEMENTS

- ❖ I explored the inventory management domain in detail.
- ❖ I gained insights and feedback for project and career growth throughout the project term from Dr. Adel Merabet.
- ❖ I developed an end-to-end pipeline for automating the robot to perform planogram and tag compliance.
- ❖ I explored and learned more about the different hardware components of the robot.

CHALLENGES

- ❖ **Planogram generation time:** the generation of a planogram for the current shelf takes an average of 35 minutes due to the lack of sturdy robot architecture. I had to extract multiple redundant frames to overcome this limitation.
- ❖ **Huge learning curve:** the entire project required learning new concepts, researching through the steps from scratch, and learning more about several hardware components and the networking between components. It was a challenge I enjoyed.

PROJECT GOALS REVIEW

Table. 11 provides the project goals completion status.

Table 10: Project Goals Completion Status

Sno	Project Goal	Status
1	Establish a planogram compliance pipeline, including detailed descriptions of each step.	Completed
2	Establish a tag compliance pipeline, including detailed descriptions of each step.	Completed
3	Implement end-to-end planogram compliance pipeline.	Completed
4	Implement end-to-end tag compliance pipeline.	Completed
5	Collaborate with Mohammed to embed the relevant Python scripts in Nvidia Jetson Nano to automate the planogram and tag compliance using the autonomous robot prototype.	Completed
6	Assist Dr. Adel Merabet in drafting content for research paper publication in a suitable conference or journal paper.	Completed

LEARNING GOALS REVIEW

Table. 12 provides the learning goals completion status.

Table 11: Learning Goals Completion Status

No	Goals	Tasks	Status
1	Planogram generation	Learn how an actual planogram is available.	Completed
		Create a planogram sketch for our customer racks in the lab.	Completed
2	Planogram generation	Apply image stitching to stitch images row-wise	Completed
		Applying vertical stitching/concatenation to generate the planogram	Completed
3	Planogram compliance	Apply object detection on the custom retail products dataset created using YOLOv5 and check if the extracted planogram complies with the actual planogram.	Completed
4	Tag compliance	Apply OCR extraction technique using Python to extract necessary information from the tags, which is needed to check if the tag information matches with the database information.	Completed
5	Insights and feedbacks	Gain valuable guidance, insights, and valuable feedback from Dr. Adel Merabet, Vinod and Mohammed to complete the project.	Completed

CONCLUSION

1. Summary

Two retail-related processes with an enormous scope of automation are planogram compliance and tag compliance. This report provides detailed information on how I completed a series of systematic processes to ensure the automation of planogram and tag compliance pipelines using an autonomous robot prototype. The Planogram compliance pipeline includes frames extraction, image stitching, image concatenation, extracting the planogram sequence from the database, object detection using Yolov5, planogram compliance check, and report generation. The tag compliance pipeline includes frames extraction, label detection, label resolution enhancement, compliance, and report generation. I got the opportunity to learn about Nvidia Jetson Nano and Nvidia Jetson Tx2. To sum up, I would cherish the learning experience under Dr. Adel, and this project would stand as a milestone in my career.

2. Work Distribution

Table 12: Hours Spent

Project Phase	Time Variables
Project Start Date	September 9, 2022
Project End Date	January 24, 2023
Hours Spent	306
Documentation Writing (Hrs)	20
Total Project Duration (Hrs)	326

FUTURE WORK

- ❖ Increase the number of cameras and perform planogram and tag compliance.
- ❖ Optimize the image stitching and planogram generation time.
- ❖ Empty spots and half-empty detection and testing.
- ❖ Test the robot in a real-time store.
- ❖ Increase the dataset variety by adding more items to the shelves.

REFERENCES

- [1] Varol, Gül, and Rıdvan Salih Kuzu. "Toward retail product recognition on grocery shelves." Sixth International Conference on Graphic and Image Processing (ICGIP 2014). Vol. 9443. SPIE, 2015.
- [2] Parameswaran, Latha, and Karthikeyan Vaiapury. "An automated vision based change detection method for planogram compliance in retail stores." Computational vision and bio inspired computing. Springer, Cham, 2018. 399-411
- [3] Minu, S., and Amba Shetty. "A comparative study of image change detection algorithms in MATLAB." Aquatic procedia 4 (2015): 1366-1373.
- [4] Hinton, Geoffrey E., Simon Osindero, and Yee-Whye Teh. "A fast learning algorithm for deep belief nets." Neural computation 18.7 (2006): 1527-1554.
- [5] Ciregan, Dan, Ueli Meier, and Jürgen Schmidhuber. "Multi-column deep neural networks for image classification." 2012 IEEE conference on computer vision and pattern recognition. IEEE, 2012.
- [6] Lawrence, S., and C. L. Giles. "Ah Chung Tsoi., and Back." AD" Face recognition: a convolutional neural-network approach," in IEEE Transactions on Neural Networks 8.1 (1997): 98-113.
- [7] Chong, Timothy, Idawati Bustan, and Mervyn Wee. "Deep learning approach to planogram compliance in retail stores." Semantic Scholar (2016): 1-6.
- [8] Girshick, Ross, et al. "Rich feature hierarchies for accurate object detection and semantic segmentation." Proceedings of the IEEE conference on computer vision and pattern recognition. 2014.

APPENDIX

Codes, reports, and planogram sketches are provided in the link below.

https://drive.google.com/file/d/1fq50myfqAaD49ze6xKZus4Wi4tEJVY_n/view?usp=share_link

GLOSSARY

1. List of Abbreviations

Table 13: List of Abbreviations

Abbreviation	Full Form
BN	Batch Normalization
CNN	Convolution Neural Network
Yolov5	You Only Look Once Version 5
CV	Computer Vision
Concat	Concatenation
ROS	Robot Operating System
OCR	Optical Character Recognition

2. List of Images



Figure 7: Storing images



Figure 8: Row-wise stitched image (Row 1)



Figure 9: Planogram generated

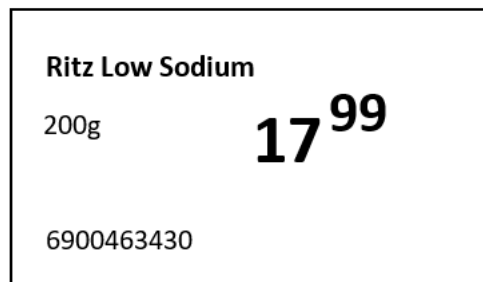


Figure 10: Custom tag